

Advanced Natural Language Processing

Lecture 02 Tokenization

Department of Advanced Computer Sciences (DACS)
Master of AI / DSDM
Prof dr ir Jan C. Scholtes – Course year 2026-2027



Maastricht University

Learning Objectives

- Distinguish between characters, tokens, terms, and words, and explain why the boundaries between them are language- and domain-dependent
- Identify tokenization edge cases (punctuation, hyphens, numbers, URLs) and explain why whitespace-only tokenization fails on them
- Explain why Chinese, Japanese, Arabic, and German each break the “words are separated by whitespace” assumption
- Compare normalization, stemming, and lemmatization, and explain what semantic information each one risks discarding
- Explain how WordPiece subword tokenization addresses the out-of-vocabulary problem, and where it can still fail (domain-specific text)



Natural Language Text: Tokenization

- Characters
- Phonemes (sounds)
- Words
- Words are used in root forms (morphemes) and in modified conjugated forms (inflections, prefix, postfix, plurals, abbreviations, etc.).

Teaching note: This character→word hierarchy is exactly what a tokenizer has to reconstruct — and morphology (root + inflection) is why “word” and “token” aren't always the same thing.

Tokenization

- Let's start looking at words
- Or should we name them terms or tokens....

Not everything we run into is a linguistic “word”...

Teaching note: Preview: “token” = any extracted unit; “term” = the normalized form used for indexing (see slide 18) — not every token is a linguistic word.



Lexicon

- By being able to look up a word, we immediately know everything about that word:
 - Is it a verb, noun, adjective, or all of the above?
 - What are the possible inflexions
 - What are possible tenses
 - What is meaning?
 - Are there any proverbial forms?

Teaching note: A lexicon only works if you already know the word's exact surface form — WordNet (next slide) adds relations between meanings, not just entries.



tends to yield to stronger personalities) — see WEAK 2
 7 not harsh or stern especially in nature or effect (a *soft* breeze coming off the lake) — see GENTLE 1
 8 providing physical comfort (a warm, *soft* bed to rest my weary bones) — see COMFORTABLE 1
 9 tolerant and kind in the judgment of and expectations for others (thought they were too *soft* with their children) — see INDULGENT 1
 10 marked by temperatures that are neither too high nor too low (It was a lovely *soft* spring evening.) — see CLEMENT 1
soften *vb* 1 to diminish the physical strength of (Three weeks of being sick in bed had noticeably *softened* her.) — see WEAKEN 1
 2 to lessen the shock of (The author's agent tried to *soften* the blow of the publisher's rejection.) — see CUSHION
softened *adj* lacking bodily strength (The athlete, *softened* by the long period of convalescence, had to begin his training program almost from scratch.) — see WEAK 1
softhearted *adj* having or marked by sympathy and consideration for others (a *softhearted* person who never hesitates to help anyone in trouble) — see HUMANE 1
softheartedness *n* 1 the capacity for feeling for another's unhappiness or misfortune (the kind of *softheartedness* that makes him an easy target for anyone with a tale of woe) — see HEART 1
 2 sympathetic concern for the well-being of others (Her *softheartedness* makes it hard for her to say "no" to charitable appeals.) — see BENIGNANCY
softness *n* the quality or state of lacking strength of will or character (Softness is not a typical quality of military leaders.) — see WEAKNESS 2
soft-soap *vb* 1 to get (someone) to do something by gentle urging, special attention, or flattery (She cunningly *soft-soaped* her parents into letting her go on the trip.) — see COAX
 2 to praise too much (shrewd voters who know when a politician is trying to *soft-soap* them) — see FLATTER 1
soft soap *n* excessive praise (a salesman who knows the value of *soft soap* in making a sale) — see FLATTERY
softy or **softie** *n* a person lacking in physical strength (a *softy* who usually needs someone else's strong hands to open bottles and jars) — see WEAKLING 1
soggy *adj* containing, covered with, or thoroughly penetrated by water (We'll spread the *soggy* papers out to dry.) — see WET 1
¹ **soil** *n* foul matter that mars the purity or cleanliness of something (I got some sort of *soil* on my white pants.) — see FILTH 1
² **soil** *n* 1 the loose surface material in which plants naturally grow (bought rich *soil* to plant flowers in) — see DIRT 1
 2 the solid part of our planet's surface as distinguished from the sea and air (happy to have *soil* under my feet after that long sea voyage) — see EARTH 2
soil *vb* to make dirty (Oil and grease *soiled* the mechanic's shirt.) — see DIRTY
soilage *n* the state or quality of being dirty (The fee for cleaning the carpet will depend upon the extent of the *soilage*.) — see DIRTINESS
soiled *adj* not clean (a *soiled* carpet in need of a good shampooing) — see DIRTY 1
sojourn *n* a temporary residing as another's guest (Alex spent a relaxing *sojourn* in her friend's summer home.) — see VISIT 1
sojourn *vb* to reside as a temporary guest (began their retirement by leisurely *sojourning* with friends and relatives scattered across the country) — see VISIT 2
solace *n* 1 a feeling of ease from grief or trouble (The kind words brought a little *solace*.) — see COMFORT 1
 2 the giving of hope and strength in times of grief, distress,

or suffering (the selfless *solace* of the sick by the workers at the hospice) — see CONSOLATION 1
solace *vb* 1 to ease the grief or distress of (Counselors did their best to *solace* the bereaved.) — see COMFORT 2
 2 to cause (someone) to pass the time agreeably occupied (I *solaced* myself with a book while I waited for the bus.) — see AMUSE
solacing *n* the giving of hope and strength in times of grief, distress, or suffering (I will be forever grateful for my friend's *solacing* of me when I was down.) — see CONSOLATION 1
solar plexus *n* the part of the body between the chest and the pelvis (felt a sharp pain in the *solar plexus*) — see STOMACH 1
soldier *n* a person engaged in military service (a platoon of *soldiers*)
synonyms dogface, fighter, legionary, legionnaire, man-at-arms, regular, serviceman, warrior
related words servicewoman; cavalier, cavalryman, cuirassier, dragon; doughboy, footman, foot soldier, infantryman; commando, raider; marine, ranger; artilleryman, cannoner; gunner, musketeer, rifleman; GI, guardsman, militiaman, minuteman; conscript, draftee, enrollee, recruit; reservist
antonyms civilian
soldierly *adj* of, relating to, or suitable for war or a warrior (Noah Webster's brief contribution to the Revolutionary cause suggests that his *soldierly* skills were few.) — see MARTIAL 1
sole *adj* 1 belonging only to the one person, unit, or group named (The landowner has *sole* rights to the property, so he can do whatever he wants to with it.)
synonyms exclusive, single, unshared
related words proprietary; personal, private
near antonyms common, communal, conjoint, cooperative, joint, multiple, mutual, pooled, public, shared, united
antonyms nonexclusive
 2 being the one or ones of a class with no other members (The eldest son became the family's *sole* support.) — see ONLY 2
solecism *n* a socially improper or unsuitable act or remark (the *solecism* of asking one's hosts how much something in their house cost them) — see IMPROPRIETY 2
solely *adv* 1 for nothing other than (The promotion was based *solely* on merit.)
synonyms alone, exclusively, just, only, purely, simply
related words basically, by and large, chiefly, generally, largely, mainly, mostly, predominantly, primarily, principally, substantially
near antonyms additionally, also, besides, likewise
 2 without aid or support (You undertook that project *solely* on your own, and you will finish it likewise.) — see ALONE 1
solemn *adj* 1 having or showing a formal and serious or reserved manner (The director of the funeral home has a fittingly *solemn* demeanor.) — see DIGNIFIED
 2 not joking or playful in mood or manner (solemn as a judge) — see SERIOUS 1
 3 causing or marked by an atmosphere lacking in cheer (The Capitol's rotunda was draped in *solemn* decorations of a state funeral.) — see GLOOMY 1
solemnity *n* 1 a mental state free of jesting or trifling (The coronation ceremony requires absolute *solemnity*.) — see EARNESTNESS
 2 an oft-repeated action or series of actions performed in accordance with tradition or a set of rules (Elaborate *solemnities* marked the 100th anniversary of the event.) — see RITE
solemnness *n* a mental state free of jesting or trifling (I laid the wreath at the Tomb of the Unknown Soldier with all the *solemnness* that the occasion requires) — see EARNESTNESS

Thesaurus or Ontology: Wordnet

- Meaning
- Associated words
(opposite, similar meaning)

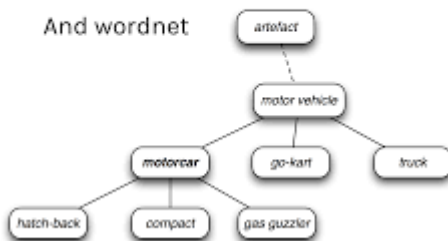
Hamburger

- Hamburger (an inhabitant of Hamburg)
 - direct hypernym:
 - German (a person of German nationality)
 - sister term
 - German (a person of German nationality)
 - East German (a native/inhabitant of the former GDR)
 - Bavarian (a native/inhabitant of Bavaria)
 - derivationally related form
 - Hamburg (a port city in northern Germany on the Elbe River that was founded by Chalemagne in the...)

<https://wordnet.princeton.edu/>



And wordnet



TOKENS AND TERMS

Tokenization

- Extracting tokens seems simple tasks:
 - Break text on whitespace characters such as TAB, SPACE or PUNCTUATIONS.
 - Non-whitespace characters are words or a token.
 - This can be done top-down and bottom-up.
- Token is a consecutive string of alphanumerical characters. Between tokens we will find non-token characters.
- But many problems can occur:
 - Mix of non-alphanumeric and alphanumeric (“Yahoo!”, “AT&T”).
 - Periods are sentence separator, but they can also be used in abbreviations and internet addresses: Mr. Smith, j.scholtes@maastrichtuniversity.nl, etc.
 - Hyphens join words but can also be used to indicate numerical ranges: x-ray versus 12-23.
 - The slash (“/”) is used in names of files and directories.
 - URL’s can have any character in them, often escaped by “%” or other non-numerical characters.
 - Phone numbers, dates, license plates, bank accounts can have any form.
 - Emoticons: ☺ or :-)).

Teaching note: Notice the common thread: every “exception” above involves a character (., -, /, %) that is sometimes a separator and sometimes part of the token.

Token Assembly aka Tokenizer

Composes lists of tokens and end locations from unstructured text streams.

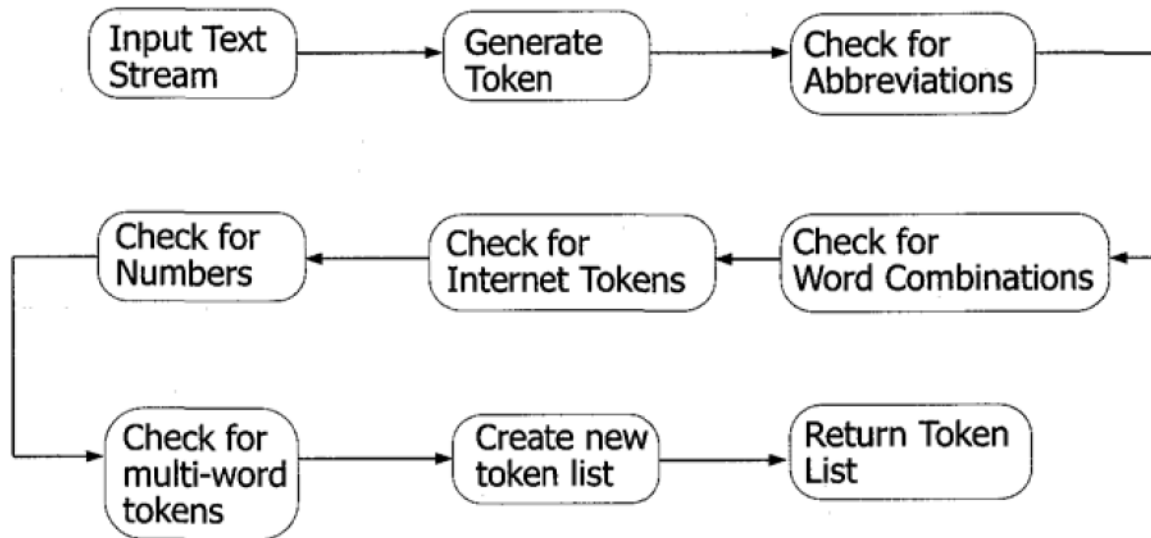


FIGURE 3.1 Steps in the assembly of composite tokens.

Example of a Tokenization Process

*Jaguar will sell its new XJ-6 model in the U.S. for a small fortune :((http://
.jaguar.com/sales or contact xj-6 @jaguar.com)*

TABLE 3.2 List of Tokens

Token	Type	Token	Type
Jaguar	Word	for	Word
Will	Word	a	Word
Sell	Word	small fortune	Collocation
Its	Word	:(	Emoticon
New	Word	(	Punctuation
XJ-6	Mixed	http://jaguar.com/sales	Internet
Model	Word	or	Word
In	Word	contact	Word
The	Word	xj-6@jaguar.com	Internet
U.S.	Collocation	)	Punctuation

Teaching note: The “Type” column shows that a tokenizer must classify, not just split: the same character class (e.g. “:”) can be an emoticon, and “xj-6@jaguar.com” must survive as one Internet token, not be split on the period or @.

Tokenization

- Issues in tokenization:
 - *Finland's capital* → *Finland? Finlands? Finland's?*
 - *Hewlett-Packard* → *Hewlett* and *Packard* as two tokens?
 - *state-of-the-art*: break up hyphenated sequence.
 - *co-education*
 - *lowercase, lower-case, lower case* ?
 - It can be effective to get the user to put in possible hyphens
 - *San Francisco*: one token or two?
 - How do you decide it is one token?

Teaching note: No single rule solves these — “correct” segmentation is a judgment call that depends on the downstream task (search vs. parsing vs. translation).

Numbers

- *3/12/91* *Mar. 12, 1991* *12/3/91*
- *55 B.C.*
- *B-52*
- *My PGP key is 324a3df234cb23e*
- *(800) 234-2333*
 - Often have embedded spaces
 - Older IR systems may not index numbers
 - But often very useful: think about things like looking up error codes/stacktraces on the web
 - (One answer is using n-grams: Lecture 2)
 - Will often index “meta-data” separately
 - Creation date, format, etc.

Tokenization: language issues

- French
 - *L'ensemble* → one token or two?
 - *L* ? *L'* ? *Le* ?
 - Want *l'ensemble* to match with *un ensemble*
 - Until at least 2003, it didn't on Google
 - Internationalization!
- German noun compounds are not segmented
 - *Lebensversicherungsgesellschaftsangestellter*
 - 'life insurance company employee'
 - German retrieval systems benefit greatly from a **compound splitter** module
 - Can give a 15% performance boost for German

The longest German Words...

Until last year:

Rindfleischetikettierungsüberwachungsaufgabenübertragungsgesetz

“Oversight Committee for the Labeling of Beef”

But in 2013 the parliament of Mecklenburg-Vorpommern decided to remove the word from the official law books, so it is no longer used.

In Germany’s most known dictionaries (Dedens), the longest word that can be found is: ***Kraftfahrzeug-Haftpflichtversicherung*** (“car liability insurance”).

Still 36 characters...

So, it is pretty obvious that fixed length data structures for dictionaries do not really work!

How about: Llanfairpwllgwyngyllgogerychwyrndrobwllllantysiliogogoch



Only the
2nd
longest
name in
the
world.

LLANFAIRPWLLGWYNGYLLGOGERYCHWYRNDROBWLLLLANTYSILIOGOGOGOGCH

ST MARYS CHURCH IN THE HOLLOW OF THE WHITE HAZEL NEAR TO THE RAPID WHIRLPOOL OF LLANTYSILIO OF THE RED CAVE

Teaching note: For a tokenizer, this is still just one token — no whitespace means no split point, regardless of length.



The longest name in the world...

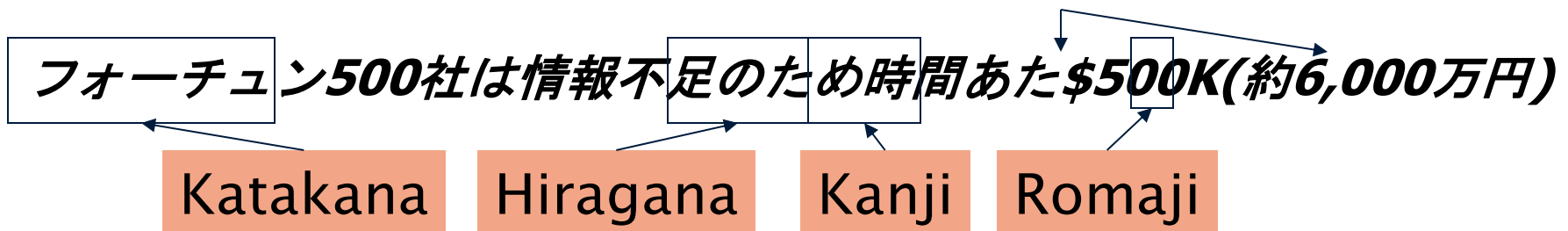


A hill near Pōrangahau, south of Waipukurau, in southern Hawke's Bay, New Zealand (meaning: "the summit where Tamatea, the man with the big knees, the slider, climber of mountains, the land-swallower who travelled about, played his kōauau (flute) to his loved one")

Tokenization: language issues

Teaching note: “Not always guaranteed a unique tokenization” = the same character string can be legally segmented more than one way — there is no whitespace oracle to resolve it.

- Chinese and Japanese have no spaces between words:
 - 莎拉波娃现在居住在美国东南部的佛罗里达。
 - Not always guaranteed a unique tokenization
- Further complicated in Japanese, with multiple alphabets intermingled
 - Dates/amounts in multiple formats



End-user can express query entirely in hiragana!

Tokenization: language issues

- Arabic (or Hebrew) is basically written right to left, but with certain items like numbers written left to right
- Words are separated, but letter forms within a word form complex ligatures

استقلت الجزائر في سنة 1962 بعد 132 عاما من الاحتلال الفرنسي.

← → ← →

← start

- ‘Algeria achieved its independence in 1962 after 132 years of French occupation.’
- With Unicode, the surface presentation is complex, but the stored form is straightforward

Normalization to terms

- We need to “normalize” words in indexed text as well as query words into the same form
 - We want to match ***U.S.A.*** and ***USA***
- Result is terms: a **term** is a (normalized) word type, which is an entry in our IR system dictionary
- We most commonly implicitly define equivalence classes of terms by, e.g.,
 - deleting periods to form a term
 - *U.S.A., USA* (*USA*)
 - deleting hyphens to form a term
 - *anti-discriminatory, antidiscriminatory* (*antidiscriminatory*)

Teaching note: This is the “term” promised on slide 3: many different tokens (U.S.A., USA) can normalize to the same indexed term.

Normalization: other languages

- Accents: e.g., French *résumé* vs. *resume*.
- Umlauts: e.g., German: *Tuebingen* vs. *Tübingen*
 - Should be equivalent
- Most important criterion:
 - How are your users like to write their queries for these words?
- Even in languages that standardly have accents, users often may not type them
 - Often best to normalize to a de-accented term
 - *Tuebingen, Tübingen, Tubingen* \ *Tubingen*

Case folding

- Reduce all letters to lower case
 - exception: upper case in mid-sentence?
 - e.g., *General Motors*
 - *Fed* vs. *fed*
 - *SAIL* vs. *sail*
 - Often best to lower case everything, since users will use lowercase regardless of ‘correct’ capitalization...
- Google example:
 - Query *C.A.T.*
 - #1 result was for “cat” (well, Lolcatz) *not* Caterpillar Inc.



Thesauri

- Do we handle synonyms?
 - E.g., by hand-constructed equivalence classes
 - ***car*** = ***automobile*** ***color*** = ***colour***
 - We can rewrite to form equivalence-class terms
 - When the document contains ***automobile***, index it under ***car-automobile*** (and vice-versa)
 - Or we can expand a query
 - When the query contains ***automobile***, look under ***car*** as well
- What about spelling mistakes?
 - Incomplete string-matching algorithms such as EDIT Distance (more in this in the Text Mining and Information Retrieval courses)
- More problematic: homonyms:
 - Bank: of a river? To put your money? To sit on? ...

Stemming

- Reduce terms to their “roots” before indexing
- “Stemming” suggest crude affix chopping
 - language dependent
 - e.g., *automate(s)*, *automatic*, *automation* all reduced to *automat*.

for example, compressed and compression are both accepted as equivalent to compress.



for exampl compress and
compress ar both accept
as equival to compress

Typical rules in Porter

- *sses* → *ss*
- *ies* → *i*
- *ational* → *ate*
- *tional* → *tion*
- Rules sensitive to the *measure* of words
- $(m > 1)$ *EMENT* →
 - *replacement* → *replac*
 - *cement* → *cement*

Teaching note: “Measure” (m) counts vowel-consonant sequences — stops rules like *EMENT* → $\emptyset$ from over-stripping short words.

Lemmatization

- Reduce inflectional/variant forms to base form
- E.g.,
 - *am, are, is* → *be*
 - *car, cars, car's, cars'* → *car*
- *the boy's cars are different colors* → *the boy car be different color*
- Lemmatization implies doing “proper” reduction to dictionary headword form

Stemming & Lexicons

Stemming

adjustable → adjust
formality → formaliti
formaliti → formal
airliner → airlin ⚠

Lemmatization

was → (to) be
better → good
meeting → meeting

	original_word	stemmed_words
0	connect	connect
1	connected	connect
2	connection	connect
3	connections	connect
4	connects	connect

Which of these two would you consider to be the better choice?

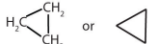
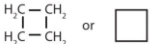
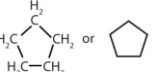
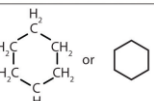
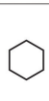
Language-specificity

- Many of the above features embody transformations that are
 - Language-specific and
 - Often, application-specific
- These are “plug-in” addenda to the indexing process
- Both open source and commercial plug-ins are available for handling these

Domain Specific Challenges

- Chemical formulas and names
- Biomolecular Sequences (DNA/RNA)
- Scientific text with embedded formulas
- OCR-noise

Generic tokenizers will destroy the information that is contained in such text strings.

Name	Molecular Formula	Structural Formula
cyclopropane	C_3H_6	 or 
cyclobutane	C_4H_8	 or 
cyclopentane	C_5H_{10}	 or 
cyclohexane	C_6H_{12}	 or 

Biomolecular sequences

DNA 5' - **ACGATCGACTGGTATATCGATGCT** - 3'
 $X_i \in \{A, C, G, T\}$

RNA 5' - **ACGAUCGACUGGUAUAUCGAUGCU** - 3'
 $X_i \in \{A, C, G, U\}$

Protein **MFINRWLFSTNHKDIGTLYLLFGAW**
 $X_i \in \{A, R, N, D, C, E, Q, G, H, I, L, K, M, F, P, S, T, W, Y, V\}$

CS369 2007

4

denote the number of nonzero coordinates of β , where $\mathbb{I}_J$ denotes the indicator function $\mathbb{I}_J(\beta) = |\{j \in \{1, \dots, M\} : \beta_j \neq 0\}|$ and $|J|$ denotes the cardinality of J . The value $\mathbb{M}(\beta)$ characterizes the *sparsity* of the vector β . The smaller $\mathbb{M}(\beta)$, the “sparser” β . For a vector $\beta \in \mathbb{R}^M$ and a subset $J \subset \{1, \dots, M\}$, we denote by β_J the vector in $\mathbb{R}^M$ that has the same coordinates as β on J and zero coordinates on the complement J^c of J .

Introduce the residual sum of squares

$$\hat{S}(\beta) = \frac{1}{n} \sum_{i=1}^n (Y_i - f_\beta(Z_i))^2$$

for all $\beta \in \mathbb{R}^M$. Define the Lasso solution $\hat{\beta}_L = (\hat{\beta}_{1,L}, \dots, \hat{\beta}_{M,L})$ by

$$(2.1) \quad \hat{\beta}_L = \arg \min_{\beta \in \mathbb{R}^M} \left\{ \hat{S}(\beta) + 2r \sum_{j=1}^M \|f_j\|_1 |\beta_j| \right\},$$

where $r > 0$ is some tuning constant, and introduce the corresponding Lasso estimator

Multa tlä@tä fyNlkÄsiii khtalostu ,ct , Abouil Asi,3 wic!lä ticiun't>t ,mitää>«, »vaalii luiftti iloista M,m<jä Tshiragauissa, ©elä fi:föf3>i'öi että uiUfatfpäim -uhkaisiloui i Hviiarat, miinto fu^tiaani 'fatifei- fuffotai» lÄuja THi roinin, puutarhassa ja, ipici'iliti hwi'tt<iioii fmmiamcrk^iUi ja anoo» »imilyMla,

Mutta tästä synkästä kohtalosta ei Abbul Asib »ielä tiennyt mitään, vaan »ietti iloista elämää TshiraganiSsa. Sekä sis>Stä »ttä ulkoapäin uhkasivat «aarat. mutta sulttaani katseli lukkotaisteluja Tfhiaaaniin puutarhassa ja palkitsi voittajan lunnicnnerleillä ja ar° vonimityksillä.

Addressing Tokenization Challenges

- Language specific tokenization
- Domain specific tokenization
- Pre-tokenization (recognize specific patterns, exceptions, language phrases, ...)
- Pre-process (normalize, substitute, add token separation, recognize sentences, ...)
- Keep original tokens (avoid irreversibility)
- Build domain specific dictionaries
- Be careful with sub-word mechanisms (we will discuss this in more detail later with the WordPiece tokenizer and word-embeddings).

Word Piece Tokenizer

- The Word Piece tokenizer is a subword tokenization algorithm widely used in modern Natural Language Processing (NLP) models, particularly in transformer-based architectures such as BERT and GPT.
- It effectively balances vocabulary size and the ability to represent rare and unseen words while improving computational efficiency.
- Word Piece is language and domain dependent.

The Word Piece Tokenizer

- BERT uses what is called a Word Piece tokenizer.
 - Words are tokenized either into the full forms if it is a known word (part of a 30k vocabulary, then one word becomes one token), or
 - Words are tokenized into word pieces (if it is not part of the 30k vocabulary), then one word can be broken into multiple tokens.

We will discuss this in more detail in the tokenization tutorial

Word	Token(s)
surf	['surf']
surfing	['surf', '##ing']
surfboarding	['surf', '##board', '##ing']
surfboard	['surf', '##board']
snowboard	['snow', '##board']
snowboarding	['snow', '##board', '##ing']
snow	['snow']
snowing	['snow', '##ing']

Advantages of Word Piece

Word Piece tokenization has several advantages:

- 1. Efficient Vocabulary Size:** Reduces the vocabulary size compared to word-level tokenization while preserving word representation quality.
- 2. Handles Out of Vocabulary (OOV) Words:** Unknown words are decomposed into known subwords, improving model robustness.
- 3. Better Morphological Representation:** Works well with morphologically rich languages by capturing subword structures.
- 4. Improves Downstream NLP Performance:** It enhances transformer-based models by ensuring that embeddings are well-distributed across different words.

Limitations of WordPiece

While WordPiece is highly effective, it does have limitations:

1. **Loss of Interpretability:** Tokenizing into subwords can make text less human-readable.
2. **Fixed Vocabulary:** Even though it handles OOV words, new words appearing in domain-specific tasks might still require retraining the tokenizer.
3. **Training Complexity:** Constructing an optimal vocabulary requires computationally expensive corpus analysis.
4. **Limited Context for Sub-Words:** too creative use of subwords may lead to non-related tokens getting similar embeddings or missing relevant contexts.
5. There is always the risks of **unknown characters, complex tokens** such as Chemical Formulas or DNA/RNA that requires special preprocessing, or specific language dependencies (direction, hyphens, 's, etc.) If such exceptions are high frequent, then such tokens will get irrelevant embeddings, similar to the random codes in a traditional tokenizer.

When Using WordPiece ...

1. Measure # of OOV tokens
2. Measure # of tokens approximated with Sub-words
3. Check for # non recognized characters
4. Check for # single or double characters and digits (or combinations)

If these # are disproportional, you may have to finetune WordPiece for your specific data set. There are already many specialized tokenizers for Chemicals, DNA, RNA, etc.

But in eCommerce applications, you may have to do something for article numbers or specific serial numbers.



Questions?



Maastricht University

Teaching note: Recap: characters → tokens → terms, each step (normalization, stemming/lemmatization, subwords) trading some precision for generalization.

Exercise 1

Consider the following multi-lingual, domain-specific text snippet: "*Prof. Müller's team at T-Mobile upgraded the Lebensversicherungsgesellschaftsangestellter-database on 12/03/2026!*"

- If you run a naive, whitespace-only tokenizer on this sentence, what specific issues arise regarding punctuation (periods, apostrophes, exclamation marks) and hyphenated compound terms?
- Explain why German compound words like *Lebensversicherungsgesellschaftsangestellter* pose a major challenge for standard fixed-vocabulary dictionaries and rule-based search engines, and why compound splitters are beneficial.
- If you apply full case folding and aggressive term normalization (e.g., stripping punctuation, hyphens, and umlauts):
 - How are "Müller's" and "T-Mobile" mapped into normalized terms?
 - What critical semantic distinction or information could be lost by doing this?

Exercise 2

Review the differences between heuristic affix-chopping (e.g., Porter Stemmer) and vocabulary/morphology-based reduction (Lemmatization)

- For each of the following words, predict the general output of a crude stemming rule versus proper dictionary lemmatization:
 - *meeting* (used in the sentence: "*The meeting was postponed.*")
 - *was* (in: "*It was an error.*")
 - *formaliti / formality*
- Is there a scenario in NLP where stemming is preferred due to speed or space saving compared to lemmatization.

Exercise 3

Modern transformers like BERT rely on subword tokenizers (such as WordPiece) to handle out-of-vocabulary (OOV) words using a fixed vocabulary (e.g., ~30k tokens).

- Using the ## prefix convention from BERT's WordPiece, show how an unseen compound word like snowboarding or surfboarding is split if the vocabulary contains snow, surf, ##board, and ##ing, but not the whole words.
- Suppose you deploy a standard pre-trained BERT WordPiece tokenizer directly on a biomedical dataset containing DNA sequences (e.g., 5'-ACGATCGACTGGT-3') or complex chemical formulas (e.g., C_6H_{12}). What failure mode occurs in terms of subword fragmentation and sequence length?
- According to the diagnostic guidelines in the lecture, what specific metrics (e.g., number of subword approximations, single/double character splits, or unrecognized tokens) should you track to determine whether you need to retrain or fine-tune a domain-specific tokenizer?

Discussion Lecture: Tokenization



Maastricht University

Teaching note: Use today's examples (German compounds, Arabic ligatures, WordPiece) as concrete cases for the abstract questions ahead.

On the Definition of a "Word" and Linguistic Universality

In Western languages, we instinctively assume a "word" bounded by whitespace is the fundamental building block of meaning.

However, languages like Chinese or Japanese omit spaces entirely, German creates continuous compounding words like *Lebensversicherungsgesellschaftsangestellter*, and Arabic blends roots with complex prefixes and ligatures.

Does the concept of an isolated 'word' actually exist as an objective universal primitive of human language, or is it an arbitrary orthographic convention that forces computational pipelines to engineer artificial solutions like compound splitters and subword segmenters?



On Normalization, Information Loss, and Downstream Semantics

In search and classical NLP pipelines, we aggressively normalize text—folding case, stripping punctuation, de-accenting characters, and reducing inflections via stemming or lemmatization to improve recall and compress the vocabulary.

But consider queries like "Fed" vs. "fed", "C.A.T." vs. "cat", or legal contracts where a hyphen or capitalization alters the entire obligation.

If every step of normalization discards syntactic or semantic signals to combat sparsity, at what point does our preprocessing turn into information destruction?

How does a modern subword tokenizer (like WordPiece) navigate this trade-off compared to a Porter Stemmer?

On Subword Tokenization: Morphological Insight or Statistical Convenience?

WordPiece solves the out-of-vocabulary (OOV) dilemma by decomposing unknown tokens into frequent substrings (e.g., surfboard → surf, ##board).

But when applied to specialized domains like chemistry, DNA sequences, or novel slang, it frequently fragments text into arbitrary character chunks and digits.

When WordPiece splits a word, is the model capturing authentic linguistic morphology (prefixes, roots, suffixes), or is it merely applying a statistical data-compression trick that happens to mimic linguistic structure?

If the latter, what are the hidden risks when these arbitrary subword pieces are fed into deep self-attention layers?